

A Project SRS
on
**BundleWise: A Retail Analytics Tool for Smart
Bundling and Profit-Driven Pricing**



Submitted by

2448005: Abishai Tom

2448033: Liya Khatija

2448036: Mansi Sapariya

Project Guide: Dr. Apash Roy

For the course

Course Code: MDS481

Course Name: PROJECT-I

1. Introduction

1.1 Purpose

The primary purpose of the BundleWise project is to develop a web-based application that enables retailers to discover smart product bundles and optimize their pricing based on purchasing patterns, forecasted sales trends, and seasonal demand. The system aims to provide an intelligent and user-friendly solution to help retailers uncover meaningful product relationships across multiple transactions, anticipate demand by analyzing past trends and external factors, and make data-informed pricing and bundling decisions to enhance customer satisfaction and business outcomes. It seeks to make advanced retail insights available to all merchants, regardless of their technical expertise, by combining automation, predictive analytics, and user-friendly design.

1.2 Scope

BundleWise is a standalone, web-based software application designed to assist retailers, especially small and medium-sized businesses, in uncovering frequently purchased product combinations (bundles) and make dynamic pricing decisions to maximize profit and improve customer satisfaction. The system analyzes scattered or incomplete transaction records, making it suitable for retailers that lack advanced infrastructure. It focuses on enabling users to upload sales data in CSV format and download actionable insights. The application provides core features, including data upload and preprocessing, wise bundling via sequential pattern mining, sales and profit forecasting, dynamic pricing optimization, an interactive dashboard, and exportable reports. The system is not embedded into any POS (Point-of-Sale) or ERP (Enterprise Resource Planning) system, and interactions primarily occur via CSV file upload/download.

1.3 Definitions, Acronyms, and Abbreviations

- **ARIMA:** AutoRegressive Integrated Moving Average - A statistical model used for time series forecasting.
- **BundleWise:** The name of the retail analytics tool for wise bundling and profit-driven pricing.
- **CSV:** Comma-Separated Values - A standard data import/export file format.
- **Dashboard:** An interactive graphical interface for visualizing insights.
- **Dynamic Pricing:** Adjusts product prices based on demand and seasonal trends.
- **ERP:** Enterprise Resource Planning - Integrated management of main business processes, often in real-time.
- **FR:** Functional Requirement - A requirement that specifies a function that a system or component must be able to perform.

- **LSTM:** Long Short-Term Memory - A recurrent neural network for time series forecasting.
- **NFR:** Non-Functional Requirement - A requirement that specifies criteria that can be used to judge the operation of a system rather than specific behaviors.
- **POS:** Point-of-Sale - The place where a retail transaction is completed.
- **Sequential Pattern Mining (SPM):** Discovers time-based purchase sequences across visits.
- **SRS:** Software Requirements Specification - A document describing a software system's requirements.

1.5 Overview of the Rest of the Document

This document is structured to provide comprehensive software requirements and Specifications for BundleWise. Section 2, “General Description,” provides an overview of the product's perspective, functions, user characteristics, general constraints, assumptions, and dependencies. Section 3, “Specific Requirements,” details the functional and non-functional requirements of the system. Section 4, “System Architecture and Design Considerations,” outlines the high-level architecture, module descriptions, and chosen technologies. Section 5, “Input and Output Requirements,” specifies the data inputs required and the various outputs generated by the system. Section 6, “Verification and Validation”, outlines the test plan and validation criteria for verifying the functional and non-functional requirements of the project. Finally, Section 7, “References,” lists all external sources cited in this document.

2. General Description

This section overviews the “**Smart Bundling and Price Optimization**” tool, outlining its system context, main capabilities, intended users, and development considerations.

2.1 Product Perspective

Smart Bundling and Price Optimization is a **standalone, web-based software application** designed to assist retailers—especially small and medium-sized businesses—to uncover frequently purchased product combinations (bundles) and make dynamic pricing decisions to maximize profit and improve customer satisfaction.

It is not embedded into any POS (Point-of-Sale) or ERP (Enterprise Resource Planning) system. Still, it works independently by allowing users to **upload sales data** in CSV format and **download actionable insights**. The system is designed to analyze scattered or incomplete transaction records, making it suitable for retailers that lack advanced infrastructure.

Using **data import/export** as the primary interaction method, the system offers retailers the flexibility to integrate insights into their existing workflows, such as inventory decisions, seasonal pricing, and promotional campaigns, without complex integrations.

2.2 Product Functions

The application provides a range of core features specifically aligned with its dual goals of **wise product bundling** and **price optimization**:

1. **Data Upload and Preprocessing**
 - Users can upload transaction data in CSV format.
 - The system automatically cleans and structures the data (e.g., date formatting, missing values handling).
2. **Smart Bundling via Sequential Pattern Mining**
 - The system identifies **frequently purchased item sequences**.
 - It analyzes time-based purchase behavior across multiple visits to find hidden product associations.
3. **Sales and Profit Forecasting**
 - Historical data is used to forecast future sales and profits using time series models like ARIMA, Prophet, or LSTM.
 - External factors like festivals or local events are included for better accuracy.
4. **Dynamic Pricing Optimization**
 - The tool suggests optimal pricing strategies for individual products and discovered bundles.

- Recommendations aim to improve sales, increase margin, and prevent overstock or understock situations.
- 5. **Interactive Dashboard**
 - Results are displayed in a user-friendly dashboard with charts and tables.
 - Users can explore bundle suggestions, trend forecasts, and pricing strategies interactively.
- 6. **Exportable Reports**
 - Users can download insights and pricing suggestions in PDF or Excel format for business use.

2.3 User Characteristics

The primary users are **non-technical retail business owners and managers**, especially from small to mid-sized firms. These users often:

- **Lack of technical expertise:** The system requires no coding knowledge.
- **Need fast and precise insights:** The dashboard is designed to be easy to understand and decision-oriented.
- **Work with incomplete data:** The system supports non-invoice or loosely structured sales records.
- **Focus on practical outcomes:** Users are more concerned with profitability and operational improvements than technical details.

The interface emphasizes ease of use, clear labeling, and guided workflows to support such users.

2.4 General Constraints

The following constraints will impact the development and usage of the system:

- **Data Quality Dependence:** The insights depend on the quality and quantity of uploaded data.
- **Algorithmic Limitations:** Forecasting and optimization models may not fully capture unexpected consumer behaviors or rare events.
- **Resource Intensive Tasks:** Some analytics functions may require high processing power, especially on large datasets.
- **Internet Access Needed:** The system is accessible via the web and relies on consistent internet access.
- **No Real-Time System Integration:** In the initial version, integration with live POS or ERP systems is not supported. All interactions happen via CSV file upload/download.

2.5 Assumptions and Dependencies

The following assumptions and dependencies are made:

- **Structured CSV Input:** It is assumed that retailers will upload data in a structured CSV format, including fields such as product ID, date of purchase, and customer ID.
- **Open-Source Tools:** The project depends on Python libraries like Pandas, NumPy, mlxtend/SPMF (for pattern mining), Facebook Prophet, TensorFlow/Keras, and Dash/Streamlit/Plotly.
- **Adequate Server Resources:** Hosting environments must support at least 6GB of RAM and CPU for data analysis and forecasting.
- **Internet Connectivity:** Users will access the system through a browser and must have stable internet access.
- **Privacy Compliance:** Uploaded data will be processed securely, and the system will follow best practices for data protection.
- **Stakeholder Feedback:** Development will rely on continuous feedback from users and stakeholders to refine system functionality and outputs.

3. Specific Requirements

3.1 Functional Requirements (FR)

3.1.1 FR1: Data Upload and Preprocessing

- The system shall allow users to upload sales data in CSV format.
- The system shall validate the uploaded file for required fields (e.g., product ID, date, quantity).
- The system shall perform data cleaning tasks such as handling missing values, duplicates, and standardizing formats.
- The system shall convert date and time information into usable time series and pattern mining formats.

3.1.2 FR2: Sequential Pattern Mining for Bundling

- The system shall extract frequent item sequences from transactional data using algorithms like PrefixSpan or SPADE.
- The system shall identify products that are frequently bought together across multiple transactions.
- The system shall present the discovered bundles with frequency metrics and confidence levels.

3.1.3 FR3: Sales and Profit Forecasting

- The system shall perform time series forecasting on individual product sales and profit margins using models like ARIMA, Prophet, or LSTM.
- The system shall incorporate seasonal components such as festivals, holidays, and local events.
- The system shall provide weekly or monthly projections of sales and profits.

3.1.4 FR4: Dynamic Pricing Optimization

- The system shall apply pricing optimization techniques to suggest price adjustments for products and bundles.
- The system shall use linear programming or heuristic approaches to maximize profit while considering demand elasticity and inventory levels.
- The system shall recommend promotional pricing strategies like discounting, upselling, and down-selling.

3.1.5 FR5: Interactive Dashboard and Visualization

- The system shall offer a user-friendly dashboard for visualizing frequent itemsets, forecasts, and pricing suggestions.
- The dashboard shall include charts, graphs, and tables for enhanced interpretability.
- The system shall support filters and drill-down options for deeper analysis.

3.1.6 FR6: Reporting and Export

- The system shall allow users to export the results of pattern mining, forecasting, and pricing in PDF or Excel format.
- The reports shall include visual summaries, actionable insights, and recommended strategies.

3.2 Non-Functional Requirements (NFR)

3.2.1 NFR1: User-Centric Design

- The system shall provide a clean, intuitive, and guided interface suitable for non-technical retail users.
- Onboarding tutorials and tooltips will help first-time users navigate the dashboard and upload the process easily.

3.2.2 NFR2: High Performance & Efficiency

- The system shall complete analysis tasks (pattern mining, forecasting, pricing suggestions) within 10 seconds for standard datasets (<50,000 records).
- Caching or background processing may be used to improve speed during repetitive tasks.

3.2.3 NFR3: Elastic Scalability

- The platform shall scale to handle increasing numbers of users and growing data volumes without performance degradation.
- It should support vertical scaling (upgrading resources) and horizontal scaling (adding servers) in cloud environments.

3.2.4 NFR4: Operational Reliability

- The system shall be available 99% of the time during working hours (8 AM – 10 PM IST).
Regular health checks and fallback mechanisms shall ensure continuity in case of component failure.

3.2.5 NFR5: Data Security and Privacy

- All user-uploaded sales data shall be stored in a secure environment, encrypted in transit (via HTTPS) and at rest.
- No data shall be shared with third parties. Temporary uploads shall auto-delete after processing (optional for privacy-sensitive users).

3.2.6 NFR6: Easy Maintainability and Upgradability

- The system's architecture shall be modular to allow component-level maintenance. Updates to forecasting algorithms, pricing models, or UI components shall be easily integrable without affecting other modules.

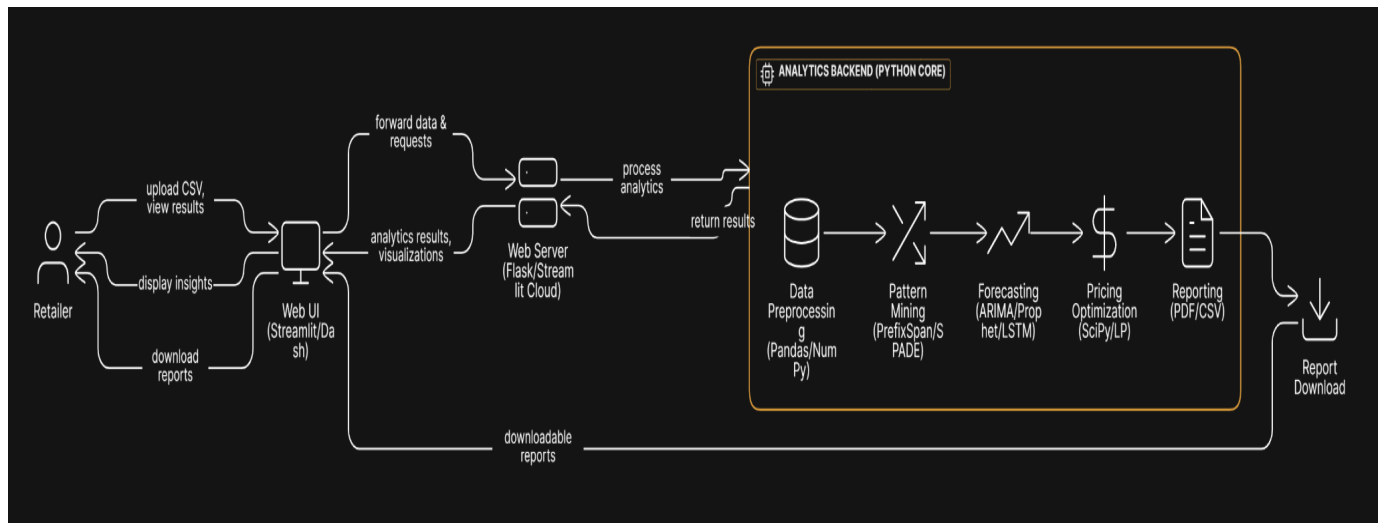
3.2.7 NFR7: Platform Independence and Accessibility

- The solution shall be web-based and compatible with major browsers (Chrome, Firefox, Safari, Edge).
- It shall support responsive design to ensure access via desktops, tablets, and smartphones.

4. System Architecture and Design Considerations

The *Smart Bundling and Price Optimization* system is designed as a **client-server web application**. The goal is to provide users (retailers) with a responsive and accessible interface while performing all data-heavy analytics on a secure backend. This architecture separates the **user-facing interface** from the **core analytics engine**, enabling scalability, ease of maintenance, and improved user experience.

4.1 High-Level Architecture Diagram



Component Description:

- **Retailer (User):**
The end user of the system who accesses the web application through a browser. The user interacts with the dashboard, uploads CSV files, and downloads analysis results.
- **Web Browser (User Interface):**
A front-end application built using **Streamlit** or **Dash**. It provides a user-friendly graphical interface for uploading data, viewing insights, and generating reports.
- **Web Server:**
This is the backend layer that serves the application to the user. It handles communication between the user interface and the analytics engine. Technologies such as **Flask**, **Streamlit Cloud**, or **Render** are used here.
- **Analytics Backend (Python Core):**
This is where all the core logic and data processing take place. It is developed using Python and handles data cleaning, analysis, forecasting, and pricing optimization.
- **Data Processing Modules:**
Use **Pandas** and **NumPy** to structure and transform the uploaded sales data.

- **Sequential Pattern Mining Module:**
To find frequently bought-together products, perform item-sequence discovery using algorithms like **PrefixSpan** or **SPADE**.
- **Forecasting Module:**
Applies predictive models such as **ARIMA**, **Prophet**, and **LSTM** to estimate future sales and profits.
- **Pricing Optimization Module:**
Uses mathematical optimization (with **SciPy**) to recommend product pricing strategies for maximizing revenue and clearing inventory.
- **Data Storage:**
Uses **CSV** files for initial input and may use temporary memory or lightweight storage (like local variables or session files) for processed data and model outputs.

4.2 Module Description

Each part of the system is organized into logical modules to ensure the separation of responsibilities and maintainability:

1. User Interface Module

- Builds a web-based dashboard that users interact with.
- Allows uploading of CSV files and downloading of results.
- Displays interactive charts, tables, and graphs for easy understanding.

2. Data Ingestion & Preprocessing Module

- Reads CSV files uploaded by users.
- Validates structure (e.g., required columns: product ID, transaction date, customer ID).
- Cleans data by removing duplicates, formatting dates, and handling missing values.

3. Sequential Pattern Mining Module

- Applies algorithms like **PrefixSpan** or **SPADE** to find frequent product purchase sequences.
- Detects product bundles purchased across multiple visits.
- Outputs the results with frequency and confidence scores.

4. Time Series Forecasting Module

- Analyzes historical sales data to forecast upcoming sales and profits.
- Uses models such as **ARIMA**, **Facebook Prophet**, and **LSTM**.
- Includes seasonal events (like festivals or holidays) to improve accuracy.

5. Pricing Optimization Module

- Suggests dynamic prices for individual products and bundles.
- Applies mathematical optimization using **SciPy** to increase profit and reduce overstock.
- Offers strategies like markdowns, discounts, and upselling.

6. Reporting Module

- Generates exportable files (CSV, PDF) that summarize:
 - Frequently bought product combinations
 - Predicted sales/profit trends
 - Recommended pricing strategies

4.3 Technologies Chosen

The project makes use of proven and robust technologies for data science, forecasting, visualization, and deployment:

Component	Technology / Library	Purpose
Programming Language	Python	Core development language due to its simplicity and rich ecosystem.
Data Handling	Pandas, NumPy	Used for data cleaning, transformation, and manipulation.
Pattern Mining	PrefixSpan / SPADE (via SPMF or mlxtend)	Discover product purchase sequences from transaction history.
Forecasting	ARIMA, Facebook Prophet, LSTM (Keras/TensorFlow)	Predict sales and profit trends over time.
Optimization	SciPy	Solve price optimization problems using numerical methods.
Dashboard/UI	Streamlit / Dash / Plotly	Build interactive and user-friendly dashboards to display insights.

Deployment	Flask, Streamlit Cloud, Render	Deploy the application to the web, flexible cloud, and local options.
File Handling	CSV	Easy-to-use format for inputting and exporting data.

4.4 Design Considerations

- **User-Focused Design:**
Simplicity and clarity are prioritized in the dashboard to accommodate non-technical users.
- **Modular Architecture:**
Each feature (e.g., bundling, forecasting, pricing) is separated into modules for easy development and maintenance.
- **Data Privacy:**
Data is handled securely during uploads, and best practices are followed to avoid unauthorized access.
- **Scalability:**
The system is built to support larger datasets in the future by using scalable libraries and deployment options.
- **Flexibility in Deployment:**
The system can adapt to different hosting environments and user needs, whether deployed locally or on cloud platforms like **Streamlit Cloud** or **Render**.

5. Input and Output Requirements

5.1 Inputs

5.1.1 Historical Sales Data

The system requires historical sales data from retailers. This data should be in a structured CSV format and include fields such as product ID, date of purchase, and customer ID.

5.1.2 Product Data (Implicit / Embedded in Sales Data)

Product data, such as product IDs and potentially product categories or descriptions, is implicitly embedded within the historical sales data provided. The system is designed to work even when the data does not contain complete invoice-level item groupings.

5.1.3 External Event Data (Optional / Configurable)

Forecasting and pricing strategies can account for external factors like festivals, weather, and special promotional events. The system can forecast based on historical data. It can include external event data as a choice or configuration option to improve accuracy and relevance.

5.2 Outputs

5.2.1 Product Bundling Suggestions

The system identifies and provides a list of products frequently bought together over time, presented as discovered bundles with frequency metrics and confidence levels. These suggestions are based on sequential pattern mining that uncovers often-purchased product sequences across multiple transactions or visits.

5.2.2 Sales and Profit Forecasts

The system computes a forecast of future sales and profits. The estimates are generally weekly or monthly, predicted with a time series model using historical data with seasonal factors and other extra factors.

5.2.3 Price Recommendations

The tool implies the best pricing strategies for individual products and found bundles. This solution seeks to maximize sales and profit and help avoid overstock and/ or understock. This solution may include promotional pricing strategies like discounting, upselling, and downselling.

5.2.4 Interactive Dashboard

All findings and recommendations are presented using an interactive and user-friendly dashboard. The report includes charts, graphs, and tables to add interpretability. The dashboard consists of a file upload feature, the ability to see trend forecasts, bundle suggestions, and report

generation. The dashboards allow users to interactively examine bundle recommendations, trend forecasting, and pricing strategies.

5.2.5 Business Reports

Users can download reports with insights and pricing recommendations in PDF or Excel format for business purposes. The reports are visual summaries that include actionable insights and recommended strategies from pattern mining, forecasting, or pricing analysis.

6. Verification and Validation

This section outlines the **test plan** and **validation criteria** for verifying the functional and non-functional requirements of the BundleWise system. It ensures the system meets the intended goals of smart bundling and dynamic pricing for retail analytics.

6.1 Test Plan Overview

The testing of **BundleWise** will include the following phases:

Phase	Purpose
Unit Testing	Test individual modules (e.g., data preprocessing, pattern mining).
Integration Testing	Test interaction between modules (e.g., forecasting with the data pipeline).
System Testing	Validate end-to-end functionality as per requirements.
Usability Testing	Ensure the interface is user-friendly for non-technical users.
Performance Testing	Assess system speed and responsiveness with real data.
Validation Testing	Confirm that the system meets user expectations and project objectives.

6.2 Validation Criteria

Validation ensures that **BundleWise** satisfies the business goals stated in the project objectives. The key validation tests and success criteria are as follows:

Test Area	Test Description	Expected Outcome
Data Upload & Preprocessing	Upload a CSV file with valid/invalid entries.	Valid files are processed correctly; invalid entries raise warnings.
Pattern Mining Module	Test with known transaction sequences.	Frequently purchased sequences are correctly identified.

Forecasting Module	Apply forecasting to synthetic and real data.	Output matches expected seasonal trends or known test results.
Pricing Optimization	Run the module on known bundle data with set constraints.	Suggested prices are optimal under given scenarios.
Interactive Dashboard	Validate all visualizations render properly and respond to filters.	Users can explore bundles, forecasts, and pricing easily.
Export/Reporting	Export forecast and bundle analysis as PDF or CSV.	The output file includes all required insights and charts.
Edge Case Handling	Upload empty files or incomplete transactions.	The system responds gracefully with appropriate error messages.

6.3 Tools Used for Testing

- **Pytest / Unittest** (Python) – for unit and integration testing
- **Selenium / Playwright** – for UI interaction testing
- **Google Collab** – for manual inspection of analysis modules
- **User Feedback Sessions** – for final acceptance

7. References

The following references were consulted during the development of this SRS:

- Debasishsaha123. (n.d.). *MARKET-BASKET-ANALYSIS*. GitHub. Retrieved from <https://github.com/Debasishsaha123/MARKET-BASKET-ANALYSIS>
- krishna-aditi.(n.d.).
product-recommendation-based-on-market-basket-analysis-and-rfm-analysis. GitHub. Retrieved from <https://github.com/krishna-aditi/product-recommendation-based-on-market-basket-analysis-and-rfm-analysis>
- ashishpatel26. (n.d.). *Market-Basket-Analysis*. GitHub. Retrieved from <https://github.com/ashishpatel26/Market-Basket-Analysis>
- mattdelaune. (n.d.). *Ecommerce_Market_Basket_Analysis*. GitHub. Retrieved from https://github.com/mattdelaune/Ecommerce_Market_Basket_Analysis
- satishrath185. (n.d.). *Market-Basket-Analysis*. GitHub. Retrieved from <https://github.com/satishrath185/Market-Basket-Analysis>
- Mjay. (n.d.). *Association Rule Mining - Market Basket Analysis*. RPubs. Retrieved from <https://rpubs.com/Mjay/1264285>